Settlement

The Compute Ledger of the Hanzo Sovereign L1 on Lux

Hanzo AI Research

Hanzo AI

2026

Abstract

Hanzo settles paid AI compute. Buyers put credit in; operators who serve attested work take rewards out. This document specifies the ledger that connects those two flows, the predicate that decides whether a served-compute claim is admitted, the bond an operator posts and the conditions under which it is reduced, and the exact boundary between what the chain records and what stays off it.

The chain is a sovereign L1 on Lux. No consensus, cryptography, or validator-set machinery is introduced here: ordering and finality are Lux Quasar, the post-quantum primitives are the Lux precompile block, the attestation verifier is the one already compiled into that block, and the L1 is born by `CreateChainTx` + `AddValidatorTx` + `ConvertNetworkTx` against the Lux P-chain and then runs on its own validator set with an on-chain manager. The contribution of this document is the accounting rules and the admission predicate, not a chain.

We separate two money flows that the current implementation braids together. A buyer's escrowed credit pays an operator for work the buyer ordered: a transfer, conserved, bounded by the order. A protocol emission additionally rewards attested work: a mint, bounded by a per-epoch pool and distributed pro-rata. Reading the deployed code, the second flow is presently unbounded — a single genuinely attested device can compute a reward of up to 6.44×10^9 AI from one claim, and can produce unboundedly many distinct claims, because the reward magnitude is read from an attacker-chosen field of the work proof and the replay key is derived from an attacker-chosen nonce. We give the specific predicate and accounting changes that close this, each expressed as a composition of primitives already present in `luxfi/precompile`: the existing ML-DSA verifier, the existing TEE device binding, and the existing spent set.

Throughout we mark what is observed in code from what is proposed, and cite the file and symbol for every claim about current behaviour.

Contents

1 Scope and method

1.1 What this document specifies

Five things, and nothing else:

(1) the ledger — pay-per-use compute credit in, AI-work reward out;

(2) operator bonds and the conditions under which a bond is reduced, including a missing or invalid attestation;

(3) the distribution of reward for served, attested compute;

(4) the admission of a served-compute claim: the attestation proof carried from the serving daemon or its TEE, the predicate that accepts it, and the settlement that follows;

(5) the boundary between on-chain and off-chain state.

1.2 What this document does not do

It does not design a chain. Ordering, finality, the validator wire, the post-quantum signature suite, the threshold suite, the TEE certificate-chain verifier, and the cross-chain message format are all Lux machinery, cited where used and not restated. It does not set token supply or an emission schedule: the published record disagrees with itself on both (§7.2), so we specify the invariant the schedule must satisfy and leave the schedule to governance. It does not specify the market that decides which operator serves which order; that is the clearing problem, treated separately.

1.3 Evidential conventions

Every statement about current behaviour is labelled and carries a file reference. **Observed** means the behaviour is present in code we read. **Derived** means it follows from observed code by an argument we give. **Proposed** means it does not exist and is put forward here. **Assumed** means it is a security or economic premise we do not prove. Repository roots: `luxfi/node`, `luxfi/precompile`, `luxfi/evm`, `luxfi/consensus` under `~/work/lux`; the off-chain plane is `hanzoai/cloud`.

The key words MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are to be interpreted as in RFC 2119.

2 The chain

2.1 Birth on the Lux P-chain

Observed. The Lux P-chain distinguishes the construction of a network from its promotion. `CreateNetworkTx` is the constructor ($\emptyset \to$ Network); `ConvertNetworkTx` is the endomorphism (Network $\to$ Network) that changes a network's security from inherited to sovereign and re-anchors its parent (`node/vms/platformvm/txs/convert_network_tx.go`). Its documentation is explicit that this is the L2-to-L1 promotion.

The security configuration carried by that transaction is a value with two orthogonal axes (`node/vms/platformvm/security/mode.go`):

```
type Mode struct {
    RestakeParent bool
    Admission Admission // NoOwnSet | Open | Gated
    Threshold uint64 // minimum stake for Open admission
    Manager Manager // PChain | Contract
```

```
}
func (m Mode) Sovereign() bool { return m.Admission != NoOwnSet }
```

`SyntacticVerify` refuses a conversion whose resulting mode is not sovereign (`ErrConvertMustEstablishOwnSet`), refuses one with no genesis validators (`ErrConvertMustHaveValidators`), and refuses `Manager == Contract` without a manager address (`ErrContractManagerNeedsAddress`).

Observed. `AddValidatorTx` is the universal entry point for adding a validator to a network; `AddChainValidatorTx` carries a deprecation notice stating that under LP-018 validators join a network and never a chain, and that "a sovereign L1 IS a primary network at its own networkID" (`add_chain_validator_tx.go`). `CreateChainTx` creates a blockchain on a network.

Specification. Hanzo is born by the sequence `CreateNetworkTx` (once, if the network does not already exist), `CreateChainTx`, `AddValidatorTx`, `ConvertNetworkTx`, executed against the upstream Lux P-chain, with

$$\texttt{Mode} = \{\ \texttt{RestakeParent} : \text{false},\ \texttt{Admission} : \texttt{Open},\ \texttt{Threshold} : \tau_{\min},\ \texttt{Manager} : \texttt{Contract}\ \}$$

and `ManagerChainID` equal to the Hanzo EVM chain, `ManagerAddress` equal to the staking contract of §6. After the conversion the L1 runs on its own validator set; the Lux P-chain is used at runtime only as the registry that mirrors that set.

2.2 Network identity

Convention. A sovereign L1 runs its own primary network whose network identifier equals its EVM chain identifier, one identifier per environment. This makes staking-key derivation paths globally unique and removes wallet ambiguity. The node's own source states the first half of this directly: "a sovereign L1 IS a primary network at its own networkID".

Observed. Two identifiers are already assigned and in use: mainnet 36963 (`hanzo-genesis.json config.chainId`; deployment records in `hanzo/node/contracts/lib/standard/DEPLOYMENTS.md`) and testnet 36962 (HIP-0024). **Proposed.** Devnet and localnet identifiers are unassigned; they MUST be allocated through the chain registry (HIP-1020) before use, and this document does not assign them.

2.3 What finality authorises

Observed. Lux finality is a ladder, not a boolean, and each rung authorises a strictly larger class of action (`consensus/engine/chain/finality.go`):

Rung	Condition	Authorises
Photon	vote in flight	nothing
Wave	forming preference	nothing
Nova	β-ignition, majority stake	local execution; reorgable
Quasar	$>2/3$ stake certificate	export: bridges, cross-chain, settlement
Horizon	post-quantum seal of that certificate	irreversible settlement

The predicates are `AuthorizesLocalExecution() { return f >= Nova }`, `AuthorizesExport() { return f >= Quasar }` and `AuthorizesIrreversibleSettlement() { return f >= Horizon }`. The stake floors have single definitions, `config.TwoThirdsStakeFloor` and `config.HalfStakeFloor`, that the certificate verifier enforces.

Specification. Each money movement in this document is bound to a rung in §9. The rule is that a movement which another party can act on irreversibly requires the rung that authorises irreversibility, and no movement is bound to a rung looser than the consequence it enables.

2.4 What is reused, and where it lives

Packages below are in `luxfi/precompile` unless the row names another repository; slot addresses are given by their significant prefix.

Concern	Mechanism	Location
Ordering, finality	Quasar	`consensus`, `engine/chain`
Signature over a claim	ML-DSA (FIPS 204)	`ai`, slot `0x012202`
TEE quote verification	X.509 chain to root	`ai/tee_attest.go`
Attestation ABI	selectors `0x01`–`0x05`	`attestation`, `0x0300...01`
Replay prevention	spent set	`ai`, `IsSpent`/`MarkSpent`
Model commitment	versioned registry	`modelregistry`, `0x0300...02`
Deterministic re-execution	integer transformer	`inference`, `0x0300...03`
Market state	provider/job records	`compute`, `0x0300...10`
Validator stake read	signed ballot	`evm`: `stakeweight`, `0x0200...06`
Cross-chain message	Warp	`evm`: `warp`, `0x0200...05`
Off-chain state commitment	anchor roots	`anchor`, `0x0700...10`

Observed. All of these except `stakeweight` are activated on Lux mainnet: `genesis/configs/mainnet/upgrade.json` lists 49 `precompileUpgrades` keys at `blockTimestamp` 1766708400, including `aiMiningConfig`, `attestationConfig`, `computeMarketConfig`, `anchorConfig`, `warpConfig`, `mldsaVerify` and `rewardManagerConfig`. `stakeWeightConfig` is implemented (`evm/precompile/contracts/stakeweight`) but does not appear in that file, so it is not active on mainnet today; §6 depends on its activation.

3 Objects

We fix notation before stating any rule. $\mathcal{B}$ is the set of buyer accounts, $\mathcal{O}$ the set of operator accounts. All amounts are integers in the smallest unit of AI (10^{-18} AI); no rule in this document uses a non-integer.

Definition 3.1 (Model commitment)**.** $\mu \in \{0,1\}^{256}$ is the identifier of a model version in the on-chain registry. **Observed:** `precompile/modelregistry` stores, per model name, a weight commitment and a version under admin control (`weightKey`, `verKey`, `IsAdmin`). μ names the pair; it does not prove that the named weights produced any particular output.

Definition 3.2 (Order)**.** $\omega = (\iota, b, \mu, \pi, \bar{m}, \delta, o)$, where ι is the order identifier, $b \in \mathcal{B}$ the buyer, μ the model commitment, π the price cap in the smallest unit, $\bar{m}$ the meter cap (the maximum quantity of work the buyer agrees to pay for, in the meter unit of §5.5), δ the deadline as a block height, and $o \in \mathcal{O} \cup \{\perp\}$ the assigned operator ($\perp$ = open to any bonded operator).

Definition 3.3 (Work proof)**.** The byte layout the AI precompile already parses (`precompile/ai/ai_mining.go`, `ParseWorkProof`):

$$\begin{aligned}&\text{deviceID}(32) \parallel \text{nonce}(32) \parallel \text{timestamp}(\text{u64})\\ &\quad \parallel \text{privacy}(\text{u16}) \parallel \text{computeMinutes}(\text{u32}) \parallel \text{teeQuote}(*)\end{aligned}$$

Definition 3.4 (Attestation envelope)**.** The envelope appended to a work proof (`precompile/ai/cross_chain_mining.go`, `verifyDeviceBinding`):

$$\begin{aligned}\text{envelope} &= \text{sigLen}(\text{u32}) \parallel \text{teeSig} \parallel \text{receipt}\\ \text{receipt} &= \underbrace{\text{deviceID}(32)\|\text{ts}(8)\|\text{nonce}(8)}_{\text{report, 48 B}} \parallel \text{certChainDER}\end{aligned}$$

Definition 3.5 (Claim)**.** $c = (\iota, o, P, \sigma, \alpha, y, m, \lambda)$: the order identifier, the operator, the ML-DSA public key P, the ML-DSA signature σ over the canonical claim bytes, the attestation envelope α, the output commitment y, the meter reading m, and the privacy level λ.

Definition 3.6 (Privacy multiplier)**.** $\phi : \lambda \mapsto \mathbb{N}$, in basis points. **Observed:** ϕ is `privacyMultipliers` in `precompile/ai/ai_mining.go`, with $\phi(1) = 2500$ (public), $\phi(2) = 5000$ (private), $\phi(3) = 10000$ (confidential) and $\phi(4) = 15000$ (sovereign); any other λ is rejected with `ErrInvalidPrivacyLevel`. We reuse the function and its domain unchanged.

Definition 3.7 (Ledger accounts)**.** Four maps, all held in chain state: $\mathrm{esc} : \mathcal{B} \to \mathbb{N}$, escrowed credit; $\mathrm{lock} : \iota \to \mathbb{N}$, credit locked against an open order; $\mathrm{bond} : \mathcal{O} \to \mathbb{N}$, bonded stake; $\mathrm{earn} : \mathcal{O} \to \mathbb{N}$, claimable earnings.

Definition 3.8 (Epoch)**.** e, a fixed count of blocks. $A(e)$ is the multiset of claims admitted in epoch e; $A_o(e)$ those attributed to operator o. $\Lambda(e)$ is the emission cap for the epoch, a governance parameter.

4 The ledger

4.1 Two flows, kept apart

Money reaches an operator by two routes with different sources, different bounds, and different failure modes.

Revenue.
A buyer's escrowed credit pays for work that buyer ordered. The source is the buyer's balance. It is a transfer: total supply is unchanged. Its bound is the order — an operator cannot be paid more than the buyer committed.

Subsidy.
The protocol emits new AI to reward attested work in aggregate. The source is issuance. It is a mint: supply increases. Its bound must be external to any individual claim, because a claim's own fields are chosen by the party being paid.

The current implementation braids them: it computes a mint from fields of the claim (§5). A mint whose magnitude is read from a field of the claim is unbounded, because the claimant writes the field. Separating the two flows is the first change this document makes.

Invariant 4.1 (Conservation)**.** For every epoch e,

$$\sum_{o \in \mathcal{O}} \Delta\,\mathrm{earn}(o) \;=\; \underbrace{\sum_{b \in \mathcal{B}} \Delta\,\mathrm{esc}(b)^{-}}_{\text{revenue, a transfer}} \;+\; \underbrace{M(e)}_{\text{subsidy, a mint}}, \qquad 0 \le M(e) \le \Lambda(e),$$

where $\Delta\,\mathrm{esc}(b)^{-}$ is the total debited from buyer b. No other rule in this document increases earn.

Invariant 4.2 (Order bound)**.** The revenue paid against order ω never exceeds π_ω, and the total revenue paid against ω over its lifetime never exceeds $\mathrm{lock}(\iota)$.

Invariant 4.3 (Bond solvency)**.** $\mathrm{bond}(o) \ge \beta_{\min}$ is a precondition of admitting any claim attributed to o. A reduction of $\mathrm{bond}(o)$ never makes it negative; a reduction is $\min(\mathrm{bond}(o), s)$ for the scheduled amount s.

4.2 Credit in

Proposed. Credit enters by an escrow deposit and leaves by withdrawal or by payment. Three transitions:

$$\begin{aligned} \textsc{Deposit}(b, v): &\quad \mathrm{esc}(b) \mathrel{+}= v, \text{ against a transfer of } v \text{ into the settlement contract} \\ \textsc{Open}(\omega): &\quad \mathrm{esc}(b) \mathrel{-}= \pi_\omega, \quad \mathrm{lock}(\iota) \mathrel{+}= \pi_\omega \\ \textsc{Withdraw}(b, v): &\quad \mathrm{esc}(b) \mathrel{-}= v \text{ if } v \le \mathrm{esc}(b) \end{aligned}$$

Locked credit is not withdrawable. An order that expires unfilled at height δ releases its lock back to the buyer: $\mathrm{lock}(\iota) \to 0$, $\mathrm{esc}(b) \mathrel{+}= \mathrm{lock}(\iota)$.

Remark on denomination. Credit is denominated in AI. Buyers who wish to hold a stable unit must do so outside the chain or through a separate instrument; introducing an oracle into the settlement path would add a trusted component to the one place where the chain's guarantee is supposed to be mechanical. This is a deliberate restriction, not an oversight, and it is a cost: it exposes buyers to AI price movement between deposit and use.

4.3 Rewards out

An operator's claimable balance is spent by Claim, which transfers $\mathrm{earn}(o)$ to o's account. **Proposed:** a withdrawal that leaves the chain (a bridge, another chain) MUST NOT execute below the Horizon rung (§9).

5 Admitting a served-compute claim

5.1 The predicate

Definition 5.1 (Admission)**.** $\textsc{Admit}(c, \omega, h)$, evaluated at block height h with block timestamp t, holds exactly when conditions A1–A7 all hold. Admission is the only event that moves money to an operator.

A1 *Form.* c parses; $|\text{workProof}| \ge 78$; the envelope's `sigLen` is nonzero and within bounds.

A2 *Authorship.* $\textsc{VerifyMLDSA}(P, \text{claim bytes}, \sigma)$. The signature binds this exact claim to the key P.

A3 *Device binding.* The receipt's certificate chain verifies to an embedded vendor root at the report's own timestamp; `teeSig` is the leaf's signature over the 48-byte report; and the attested device identity equals $\mathrm{BLAKE3}(P)$.

A4 *Freshness and quote binding.* The report timestamp t_r satisfies $t - \Delta_{\text{fresh}} \le t_r \le t + \Delta_{\text{skew}}$, and the report nonce equals $\mathrm{BLAKE3}(\iota \,\|\, \text{chainID})$.

A5 *Order binding.* ω exists, is unfilled, $h \le \delta_\omega$, $o = o_\omega$ or $o_\omega = \bot$, and the claim's model commitment equals μ_ω.

A6 *Uniqueness.* $\text{workID} = \mathrm{BLAKE3}(\iota \,\|\, \text{deviceID} \,\|\, \text{chainID})$ is unspent.

A7 *Bond.* $\mathrm{bond}(o) \ge \beta_{\min}$ and o is not jailed at h.

5.2 Which of these exist

Observed. A2 and A3 are implemented, together, in `precompile/ai/cross_chain_mining.go`. `verifyAndMintWork` performs, in order: an ML-DSA verification over the work proof; `verifyDeviceBinding`, which runs `verifyAttestation` against the embedded root pool and then checks attestedDevice = BLAKE3(P); and a third check that the work proof's own deviceID field equals BLAKE3(P). The verifier is deterministic by construction — the chain is checked at the report's embedded timestamp rather than the wall clock, and the root pool is explicit rather than the host store (`precompile/ai/tee_attest.go`, `tee_roots.go`) — and it fails closed: an empty or non-terminating chain cannot verify.

A6 exists in a weaker form: `ComputeWorkId(deviceId, nonce, chainId)` keyed on the work proof's nonce field, with `IsSpent`/`MarkSpent` over a `BLAKE3("spnt" || workId)` slot.

A1 is implemented. A4, A5 and A7 do not exist.

5.3 Why the gaps matter

Observation 5.1 (The reward magnitude is written by the party being paid)**. Observed.** `CalculateReward` reads `privacy` and `computeMinutes` directly out of the work proof and returns

$$\texttt{reward} = 10^{18} \cdot \texttt{computeMinutes} \cdot \frac{\texttt{multiplier}}{10000},$$

with `multiplier` $\in \{2500, 5000, 10000, 15000\}$ (`privacyMultipliers`). `mintWork` calls it after the spent check and returns the amount for the caller to mint. **Derived.** Neither field is covered by the TEE report — `verifyDeviceBinding` constrains only the attested device identity and the signing key — so both are chosen freely by whoever holds the ML-DSA key. `computeMinutes` is a `uint32`, so its maximum is 4,294,967,295; at the `PrivacySovereign` multiplier the single-claim maximum is 6,442,450,942.5 AI. That exceeds the total supply stated in HIP-0001 by more than six times.

Observation 5.2 (The replay key is also written by the claimant)**. Observed.** The spent-set key is BLAKE3(deviceID‖nonce‖chainID), and `nonce` is `workProof[32:64]`, a field of the claim. The report's own nonce, `receipt[40:48]`, is never compared to it. **Derived.** One attested device can produce an unbounded family of distinct, individually valid claims by varying that field; the spent set bounds replay of an identical claim, not issuance of fresh ones. Combined with Observation 5.1, a single genuine TEE device is sufficient to mint without bound.

Observation 5.3 (One quote serves forever)**. Observed.** `verifyAttestation` checks the certificate chain's validity *at the report's own timestamp*, which is what makes it deterministic, and nothing compares that timestamp to the block. **Derived.** A quote produced once remains admissible indefinitely, and remains admissible after the device has been decommissioned or its key exfiltrated. A4 is the fix: the block-relative window is the liveness check the deterministic verifier deliberately does not perform, and it belongs at the admission site, where a deterministic block timestamp is available — the attestation precompile already threads exactly that value (`accessibleState.GetBlockContext().Timestamp()`) and states the rule "never `time.Now()`".

Observation 5.4 (The compute market's verification is a tautology)**. Observed.** `precompile/compute` at `0x0300...0010` exposes `VerifyCompute(jobID, attestation)`, whose body recomputes keccak256(jobID‖outputHash) from its own storage and compares it to the supplied `attestation` (`compute/contract.go`). **Derived.** The comparand is a function of values the caller already supplied through `ClaimReward`; any caller can compute it. The check establishes nothing about a TEE, a device, or an execution. `ClaimReward` itself requires only that the caller's provider nonce is nonzero — that is, that the caller once called `RegisterProvider` — and transfers no value. **Specification.** A claim MUST be admitted by ADMIT; `compute`'s

`VerifyCompute` MUST NOT be used as an admission decision. The two real verifiers in the block — `ai.VerifyTEE` and the `attestation` adapter over it, which the latter's package documentation already names as "exactly one way to verify a TEE quote across the block" — are the only ones a settlement decision may consult.

5.4 Settlement

Algorithm 1 Settle a claim. Executes only if $\textsc{Admit}(c, \omega, h)$ holds.

1: $u \leftarrow \min(m_c,\ \bar{m}_\omega)$ ▷ meter bounded by the order
2: $\rho \leftarrow \min(\pi_\omega,\ \text{tariff}(\mu_\omega) \cdot u)$ ▷ revenue
3: $\text{lock}(\iota) \mathrel{-}= \rho$; $\quad \text{earn}(o) \mathrel{+}= \rho$
4: $\text{esc}(b) \mathrel{+}= \text{lock}(\iota)$; $\quad \text{lock}(\iota) \leftarrow 0$ ▷ refund the unspent cap
5: $W_o(e) \mathrel{+}= \phi(\lambda_c) \cdot u$ ▷ accrue the epoch weight, §7
6: $\textsc{MarkSpent}$(workID)
7: mark ω filled; record $(y_c,\ o,\ h)$

Steps 1–4 are the whole of the revenue flow, and they touch no issuance. Step 5 is the operator's only influence on the subsidy flow, and it is an addend to a share of a fixed pool, not an amount.

Proposition 5.1 (Revenue is bounded by the buyer's own commitment)**.** *Under A5 and A6, the total revenue an operator can obtain from order ω is at most π_ω.*

Proof. A5 requires the order to be unfilled and step 7 marks it filled, so at most one claim settles per order. Step 2 caps ρ at π_ω, and step 3 debits $\text{lock}(\iota) = \pi_\omega$, which $\textsc{Open}$ funded from $\text{esc}(b)$. A6 additionally prevents two claims from sharing a workID within one order, since ι is in the key. □

Remark 5.1 (Why A6 changes the key)*.* Deriving the workID from ι rather than a claim-chosen nonce converts the spent set from a duplicate filter into a one-claim-per-order rule. The change is one line at the call site of `ComputeWorkId`; the spent-set machinery, including its `BLAKE3` slot derivation and its atomic check-compute-mark ordering (which the current code correctly notes avoids a check-then-act race), is reused unchanged.

5.5 The meter

Proposed. The meter unit is the same quantity the off-chain plane already records per request, so that the number the chain pays on and the number the invoice shows have one definition. **Observed:** the off-chain recorder's wire type is deliberately narrow — subject, namespace, an exact decimal amount, currency, model and provider — and carries no prompt content and no PII (HIP-1313). The chain reads m, the meter reading, from the claim and clamps it to $\bar{m}_\omega$; the buyer set $\bar{m}_\omega$ when opening the order.

Assumed. The meter is not attested. A TEE quote of the form verified here binds a device identity and a key; it does not bind a token count or an elapsed time. Clamping to the buyer's cap is therefore the whole of the protection on the revenue side, and it is sufficient there precisely because the buyer chose the cap. It is *not* sufficient on the subsidy side, which is why the subsidy is a pool share (§7) rather than a function of m alone. Binding a meter to an execution requires either deterministic re-execution (`precompile/inference` makes this possible for the models it covers) or a proof system; both are out of scope here and named as open work in §12.

6 Operator bonds

6.1 Where the bond lives, and where it does not

Observation 6.1 (The P-chain holds no slashable bond for a sovereign L1)**. Observed.** `RegisterL1ValidatorTx` carries a `Balance` field documented as the validator's funding, and `IncreaseL1ValidatorBalanceTx` is documented as topping up "an L1 validator's *continuous-fee* balance". On removal, the executor refunds the remainder to the registered `RemainingBalanceOwner` as a UTXO (`txs/executor/standard_tx_executor.go`). **Derived.** That balance pays for the validator's continued presence; it is refundable and is not a bond against misbehaviour. **Observed.** The consensus package that detects equivocation states the division explicitly: "Actual stake deduction happens in the platformvm; this package provides detection and evidence only" (`consensus/core/slashing/slashing.go`). **Observed.** We found no stake-deduction path in `vms/platformvm/txs/executor`.

Specification. The bond is held by the Hanzo staking contract — the manager named in `ConvertNetworkTx` — on the Hanzo EVM. The Lux P-chain holds a *mirror* of the validator set's weights, not the value behind them. This is the direct consequence of choosing `Manager: Contract`, and it is the correct place for the bond: the faults that matter here are settlement faults, and settlement state is on the Hanzo EVM.

6.2 Weight is a projection of the bond

Observed. The manager changes a validator's weight by sending a Warp `L1ValidatorWeight` message — (validationID, nonce, weight), 49 bytes — which the P-chain applies through `SetL1ValidatorWeightTx`. The executor verifies the message's nonce is at least the validator's `MinNonce`, and verifies via `verifyL1Conversion` that the message came from the L1's registered manager chain and manager address. Two behaviours are load-bearing:

1. `weight == 0` is *removal*, not suspension: the executor refunds the remaining fee balance and deletes the validator, and refuses if this would remove the last validator (`errRemovingLastValidator`).

2. `nonce == MaxUint64` is reserved for removal and is rejected with any nonzero weight (`L1ValidatorWeight.Verify`, `ErrNonceReservedForRemoval`).

Specification. Weight is a monotone function of the bond, $w_o = \lfloor \text{bond}(o)/\tau_{\min} \rfloor$, recomputed by the manager whenever the bond changes and pushed as a weight message. **Proposed:** jailing MUST be expressed as exclusion inside the manager and, on the P-chain, as a weight *reduction* to a positive floor — never as weight zero, which is irreversible and requires re-registration with a fresh `RegisterL1ValidatorTx` and proof of possession. A design that jails by sending weight zero silently converts a temporary penalty into permanent ejection.

6.3 Reading stake from inside the EVM

Observed. `evm/precompile/contracts/stakeweight` at `0x0200...0006` lets a contract read a validator's P-chain weight and the set's total weight at a height, via a BLS-signed `Ballot` (168 bytes: nodeID, voter, authEpoch, P-chain height, weight, total weight, signature). The signed statement separates chain, node, voter and epoch, so one authorisation cannot be replayed onto another chain, node, address or revoked epoch; height and weight are deliberately *not* signed and are checked against P-chain state instead, which is what lets one authorisation serve every later snapshot. The expensive work is paid in `PredicateGas` before execution; the in-execution read is 2,000 gas.

Specification. The settlement contract reads participation weight through this precompile rather than trusting a self-reported figure. It therefore depends on `stakeWeightConfig` being activated, which it is not today (§2).

6.4 Faults and the schedule

F1 *Non-delivery.* An order assigned to o reaches its deadline δ with no admitted claim. Anyone may submit the evidence, which is the order identifier and the height; the chain verifies the absence itself from its own state. Reduction $s_1 = \min(\text{bond}(o), \kappa_1 \pi_\omega)$.

F2 *Invalid attestation.* A claim attributed to o satisfies A2 — so authorship is attributable to a key bound to o — and fails A3 or A4. The evidence is the claim itself. Reduction $s_2 = \min(\text{bond}(o), \kappa_2 \pi_\omega)$, rate-limited to at most n_2 occurrences per epoch per operator.

F3 *Output disagreement.* For an order whose model commitment names a deterministic profile, a challenger re-executes and submits an output commitment $y' \neq y$ together with the re-execution witness; the chain adjudicates by the deterministic path. Reduction $s_3 = \min(\text{bond}(o), \kappa_3 \pi_\omega)$ with $\kappa_3 > 1$.

F4 *Consensus fault.* Double-vote, double-sign or downtime, as classified by `consensus/core/slashing` (`DoubleVote`, `DoubleSign`, `Downtime`, with an `Evidence` record carrying the serialized conflicting votes and a `SlashRecord` carrying a jail expiry). Reduction s_4, a fixed fraction of the bond.

Parameter 6.1 (Slash coefficients)**.** $\kappa_1, \kappa_2, \kappa_3, n_2, s_4, \beta_{\min}, \tau_{\min}, \Delta_{\text{fresh}}, \Delta_{\text{skew}}$ are governance parameters. This document fixes their *constraints*, not their values.

Invariant 6.1 (Deterrence)**.** $\kappa_1 \geq 1$: the penalty for accepting an order and not serving it must be at least the revenue foregone by serving it, or non-delivery is free whenever the operator finds a better use of the device. $\kappa_3 > 1$: the penalty for a wrong answer must exceed the payment for a right one, or fabricating output is profitable at any nonzero detection probability below one.

Remark 6.1 (F2 is about attribution, not loss). An invalid attestation never settles — ADMIT rejects it — so the ledger loses nothing. F2 exists because submitting invalid attestations is a cheap way to consume verification gas and to probe the verifier, and because A2 makes the submission attributable to a bonded party. Its reduction should be small and rate-limited; treating it like F3 would punish a misconfigured operator as harshly as a lying one.

Remark 6.2 (What F1 can and cannot see). F1 is decidable from chain state alone: the order exists, the height has passed, no claim was admitted. It does not distinguish an operator who refused from one whose network failed. That is a deliberate limitation of a mechanism that must be adjudicated by a chain with no view of the operator's environment, and it argues for κ_1 near its lower bound rather than far above it.

7 Distributing reward for served, attested compute

7.1 Pool share, not per-claim mint

Proposed. At the close of epoch e, each operator's subsidy is its share of a fixed pool:

$$S_o(e) \;=\; \left\lfloor \Lambda(e) \cdot \frac{W_o(e)}{W(e)} \right\rfloor, \qquad W_o(e) = \sum_{c \in A_o(e)} \phi(\lambda_c)\, u_c, \qquad W(e) = \sum_{o \in \mathcal{O}} W_o(e),$$

with u_c the order-clamped meter of §5.5 and ϕ the privacy multiplier of §3. If $W(e) = 0$ then $S_o(e) = 0$ for all o and $M(e) = 0$: an epoch with no admitted work mints nothing. The remainder $\Lambda(e) - \sum_o S_o(e)$, at most $|\mathcal{O}| - 1$ units lost to the floors, is not minted.

The multiplier values and the basis-point arithmetic are the deployed ones. What changes is the quantity they multiply — the order-clamped meter rather than the claim's own `computeMinutes` — and the fact that the product is a weight rather than an amount.

Proposition 7.1 (The mint is bounded regardless of claim contents)**.** $M(e) = \sum_o S_o(e) \le \Lambda(e)$ *for every epoch, for every set of admitted claims, whatever values the claims carry in their meter and privacy fields.*

Proof. Each $S_o(e)$ is a floor of $\Lambda(e) \cdot W_o(e)/W(e)$ with $W_o(e) \ge 0$ and $\sum_o W_o(e) = W(e)$; the unfloored terms sum to exactly $\Lambda(e)$ when $W(e) > 0$, and flooring only decreases them. The claim's fields influence the *ratio* W_o/W and therefore the split between operators, never the total. □

Remark 7.1 (What a pool share does not fix). It bounds inflation; it does not make the split honest. An operator that inflates u takes share from operators that do not, and the clamp $u \le \bar{m}_\omega$ bounds that inflation only to what some buyer was willing to pay for. Under A5 an inflated u therefore costs the inflater real escrowed credit, which is the mechanism that makes the share approximately truthful — not the attestation. We state this plainly because it is the load-bearing economic assumption of the design and it is not proved here: it requires that a self-dealing buyer-operator pair cannot recycle credit at a profit, which holds only if the revenue leg is a genuine transfer with a nonzero cost (fees, or the opportunity cost of locked credit). See §12.

7.2 Where the emission schedule comes from

Observed. The published record disagrees. HIP-0001 states a total supply of 1,000,000,000 AI with an emission of 100M in year one halving every four years to a terminal 6.25M. The `hanzo-ai-chain` paper in this repository states a hard maximum nominal supply of two trillion, split one trillion to the DAO and one trillion to proof-earned rewards on a Bitcoin-shaped geometric halving. HIP-0024 specifies a base staking reward of 8% APY with a 2–5% AI-mining bonus, and names the consensus engine `nova`, which is a rung of the Lux finality ladder rather than an engine selector.

Specification. This document does not resolve that disagreement. It requires only that $\Lambda : e \to \mathbb{N}$ is a total function fixed in chain state before epoch e opens, that it is monotone non-increasing in e unless changed by governance, and that $\sum_e \Lambda(e)$ converges if a supply cap is claimed. The settlement rules of §4 and §7 hold for any Λ satisfying these conditions, so resolving the disagreement is a governance question and not a precondition for implementing them.

8 The on-chain / off-chain split

8.1 The rule

> The chain holds money and commitments. It never holds content. A number the chain pays on MUST be derivable from an on-chain order and a verified attestation; it MUST NOT come from an off-chain assertion alone.

8.2 The split

State	Where it lives	Why
Escrow, lock, bond, earnings	Hanzo EVM, settlement contract	it is money
Order (buyer, μ, π, $\bar{m}$, δ)	Hanzo EVM	the bound on payment
Admission decision	Hanzo EVM, calling `0x0300...`	it moves money
Spent set	precompile state	replay is a consensus property
Weight mirror	Lux P-chain, via Warp	the validator set is Lux's registry
Model commitment	`modelregistry`	a 32-byte identifier
Slash evidence and record	Hanzo EVM	adjudication must be public
Request and response payloads	off chain, never committed	content, PII
Model weights	off chain; only μ on chain	size
Per-request meter detail	`hanzo.cloud_usage` warehouse	volume
Invoices, balances, tax	commerce ledger (HIP-1313)	not consensus
Pay-per-request rail	x402 v2 (HIP-1163)	an HTTP rail, not a chain rail
Matching and scheduling	clearing plane	an optimisation, not a ledger
TEE quote generation	the operator's device	by construction
Fleet inventory, kubeconfigs	KMS-sealed, per org (HIP-0121)	secrets

8.3 The one bridge between the halves

Observed. `precompile/anchor` at `0x0700...0010` stores (appID, height, root) with per-appID height monotonicity and a 25,400-gas `submit`; `get`, `getLatest` and a range-capped `list` read it back.

Specification. The off-chain usage ledger commits to the chain by anchoring the Merkle root of each epoch's records under a fixed appID. This gives a buyer a way to prove that a detail record was in the set the operator committed to, and gives a dispute a fixed reference. **It does not make the off-chain records authoritative for payment**: payment is decided by ADMIT against the order, and the anchor is evidence for reconciliation, not an input to settlement. An anchor whose root were an input to settlement would be a trusted oracle, since the party that computes the root is the party being paid; keeping it out of the payment path is what makes it a commitment instead.

9 Binding money movements to finality

Movement	Required rung	Reason
DEPOSIT, OPEN, lock release	Nova	local, reversible with the block
Admission and earn credit	Quasar	a counterparty will act on it
Bond reduction (F1–F4)	Quasar	it is adverse to a third party
Weight message to the P-chain	Quasar	it is an export
Operator withdrawal off chain	Horizon	irreversible
Cross-chain claim of earnings	Horizon	irreversible

Derived. The middle rows follow from the ladder's own statement that a block below Quasar

can never be exported as deterministically final to a bridge, another chain, or an irreversible settlement, since Nova is reorgable by a lone equivocator holding a bare majority. The last two rows follow from `AuthorizesIrreversibleSettlement`, which requires the post-quantum seal.

Remark 9.1. Admission is placed at Quasar rather than Horizon deliberately. Admission credits an internal balance; it does not release value from the chain. Requiring the post-quantum seal for every claim would push settlement latency to the sealing cadence for no security gain, because the balance cannot leave without crossing the Horizon requirement anyway.

10 Threat model

10.1 Assets

Escrowed buyer credit; operator bonds; the issuance right (the ability to increase supply); the integrity of the validator set; and buyer request content, which the chain never holds.

10.2 Adversaries

A device holder.
Controls a genuine, vendor-certified TEE and its attestation key. Can produce arbitrarily many valid quotes for that device. *This is the adversary the current implementation does not survive* (Observations 5.1, 5.2).

A registered operator.
Holds a bond and serves orders. May under-serve, fabricate output, or inflate the meter.

A self-dealing pair.
A buyer and an operator under one principal, recycling credit to farm subsidy share.

A validator minority.
Below the Quasar floor. May equivocate; cannot export.

A network adversary.
Delays or reorders messages; cannot forge a certificate chain or an ML-DSA signature.

10.3 Assumed

The embedded vendor roots are the correct trust anchors and their private keys are not compromised. ML-DSA at the deployed parameter set is unforgeable. A TEE that verifies has not had its attestation key extracted. Fewer than one third of stake is Byzantine. None of these is proved here; the first is the weakest, because a single vendor root compromise admits arbitrary claims, and it is why A7 (a bond) is a precondition of admission rather than an alternative to attestation.

10.4 Explicitly not covered

Confidentiality of the request payload against the operator; the operator sees it by construction unless the model runs inside the TEE, which the attestation verified here does not establish. Correctness of the model weights behind μ. The economics of the clearing that assigns orders.

11 Activation

Proposed. The changes this document requires, in dependency order:

1. Activate `stakeWeightConfig` in the Hanzo chain's upgrade configuration; the module is registered (`evm/precompile/registry/registry.go`) and implemented but absent from the Lux mainnet `upgrade.json`.

2. Add the four missing admission conditions. A4 needs the block timestamp already threaded into the attestation precompile and a nonce comparison; A5, A6 and A7 need the order and bond state that the settlement contract holds. A6 is a change of argument to `ComputeWorkId`, not new machinery.

3. Deploy the settlement contract and register it as the manager address in `ConvertNetworkTx`; until then the L1 cannot be sovereign with `Manager: Contract`.

4. Change the subsidy path from a per-claim mint to an epoch pool share. `CalculateReward` remains useful as the weight function $\phi \cdot u$; its output stops being an amount and becomes an addend.

5. Retire `compute.VerifyCompute` as an admission decision (Observation 5.4). The market records in `precompile/compute` may remain as an index; they must not gate money.

Each of steps 2, 4 and 5 is a behaviour change to an activated mainnet precompile and therefore requires a new activation timestamp rather than an in-place edit.

12 Limitations and open questions

1. *The meter is unattested.* Nothing in the verified quote binds a token count or an elapsed time. The revenue side is protected by the buyer's cap; the subsidy side is protected only by the cost of the revenue leg. Closing this properly requires deterministic re-execution — `precompile/inference` is a byte-exact integer transformer built for exactly this, with the CPU/Metal/CUDA/HIP parity treated as a release conformance test — or a succinct proof. Neither is specified here.

2. *Self-dealing is bounded but not eliminated.* We assert that a buyer-operator pair pays a real cost to recycle credit; we do not derive the condition under which subsidy share exceeds that cost. This needs the fee and Λ parameters to be fixed and then measured, not argued.

3. *Only one vendor root is embedded.* `tee_roots.go` contains the Intel SGX Provisioning Certification Root CA and states that chains presenting another vendor's leaf fail closed until that vendor's root is appended. HIP-0024's validator requirements name NVTrust GPU attestation. **Derived:** GPU quotes do not verify today. Adding a root is a consensus change and must go through activation.

4. *F3 needs an adjudication procedure.* We give the fault and the penalty constraint; the challenge protocol — who may challenge, over what window, at what deposit, and how the re-execution is bounded in gas — is not specified.

5. *Jail semantics on the P-chain.* We specify jail as a weight reduction to a positive floor because weight zero is removal. The interaction between a jailed operator's reduced weight and `errRemovingLastValidator` in a small validator set is not analysed.

6. *Credit is denominated in AI.* Buyers carry price exposure between deposit and use. A stable-denominated credit requires a price input in the settlement path, which we declined; whether that trade is right is an open question and depends on buyer behaviour we have not measured.

13 Summary of evidence

File paths below are given by their leaf; the full paths are in the references.

Claim	Source	Status
L1 promotion is `ConvertNetworkTx`, own set required	`convert_network_tx.go`	observed
Security mode has two orthogonal axes	`security/mode.go`	observed
Weight zero is removal, not suspension	`standard_tx_executor.go`	observed
P-chain validator balance is a fee balance	`increase_..._balance_tx.go`	observed
Consensus detects faults, does not deduct stake	`core/slashing`	observed
Export needs Quasar; irreversibility needs Horizon	`finality.go`	observed
TEE verification is real and fails closed	`ai/tee_attest.go`	observed
Device binding is deviceID = BLAKE3(P)	`cross_chain_mining.go`	observed
Only the Intel SGX root is embedded	`ai/tee_roots.go`	observed
Reward reads attacker-chosen `computeMinutes`	`ai/ai_mining.go`	observed
Single-claim maximum 6.44×10^9 AI	`uint32` $\times$ 1.5	derived
Spent-set key uses an attacker-chosen nonce	`ComputeWorkId` site	observed
`compute.VerifyCompute` is caller-computable	`compute/contract.go`	observed
`stakeWeightConfig` is not activated on mainnet	`upgrade.json`	observed
Admission predicate A1–A7	this document	proposed
Two-flow ledger and pool-share subsidy	this document	proposed
Fault classes F1–F4 and their constraints	this document	proposed

References

[1] Lux P-chain transaction set. `luxfi/node`, `vms/platformvm/txs`: `create_network_tx.go`, `create_chain_tx.go`, `add_chain_validator_tx.go` (deprecation notice), `convert_network_tx.go`, `register_l1_validator_tx.go`, `set_l1_validator_weight_tx.go`, `increase_l1_validator_balance_tx.go`, `disable_l1_validator_tx.go`.

[2] Lux network security model. `luxfi/node`, `vms/platformvm/security/mode.go`; LP-018, network security modes.

[3] Lux Warp validator messages. `luxfi/node`, `vms/platformvm/warp/message`: `chain_to_l1_conversion.go`, `l1_validator_registration.go`, `l1_validator_weight.go`.

[4] Lux finality ladder. `luxfi/consensus`, `engine/chain/finality.go`, `engine/chain/cert.go`, `config` stake floors, `conformance/corpus.json`.

[5] Equivocation detection. `luxfi/consensus`, `core/slashing`.

[6] AI mining precompile. `luxfi/precompile`, `ai`: `ai_mining.go`, `tee_attest.go`, `tee_roots.go`, `cross_chain_mining.go`.

[7] Attestation precompile. `luxfi/precompile`, `attestation`.

[8] Compute market precompile. `luxfi/precompile`, `compute`.

[9] Model registry and deterministic inference. `luxfi/precompile`, `modelregistry`, `inference`.

[10] Anchor precompile. `luxfi/precompile`, `anchor`.

[11] Stake-weight precompile. `luxfi/evm`, `precompile/contracts/stakeweight`.

[12] Post-quantum precompile block, slots `0x012201`–`0x012208`. `luxfi/precompile`: `mlkem`, `mldsa`, `slhdsa`, `pulsar`, `p3q`, `corona`, `magnetar`, `hqc`; LP-4200.

[13] Mainnet precompile activation. `luxfi/genesis`, `configs/mainnet/upgrade.json`.

[14] HIP-0001, AI — Hanzo's native currency.

[15] HIP-0024, Hanzo Sovereign L1 Chain Architecture.

[16] HIP-0096, AI Compute Contribution Rewards.

[17] HIP-0121, BYO Compute Fleet & Metered Billing.

[18] HIP-1020, Chain Registry.

[19] HIP-1163, x402 — Pay Per Request.

[20] HIP-1313, Usage — The Metered Record.

[21] *Clearing Heterogeneous Compute*, Hanzo AI Research, this repository.

[22] *Hanzo AI Chain (AIVM)*, Hanzo AI Research, this repository.